

R2.01 - Développement Orienté Objets

Décrire des classes en UML

Arnaud Lanoix Brauer
Arnaud.Lanoix@univ-nantes.fr



Nantes Université

Département informatique

- 1 Courte introduction à UML
- 2 Modéliser des classes grâce à UML

UML = Unified Modeling Language

- UML est un **langage** de modélisation **visuel** à base de pictogrammes permettant de représenter l'ensemble des facettes d'un projet (informatique).
- UML définit un ensemble de **notations graphiques** permettant de décrire tous les concepts utiles lors des différentes étapes d'analyse et de conception (orientée objet).
- <https://www.uml.org/>



ATTENTION : orthographe des pictogrammes

UML est un **langage** comme le français : il y a donc un "orthographe" et une "grammaire".

On n'utilise pas un □ à la place d'un ◇ ou une → à la place d'une -->

UML = Unified Modeling Language

- UML est un **langage** de modélisation **visuel** à base de pictogrammes permettant de représenter l'ensemble des facettes d'un projet (informatique).
- UML définit un ensemble de **notations graphiques** permettant de décrire tous les concepts utiles lors des différentes étapes d'analyse et de conception (orientée objet).
- <https://www.uml.org/>



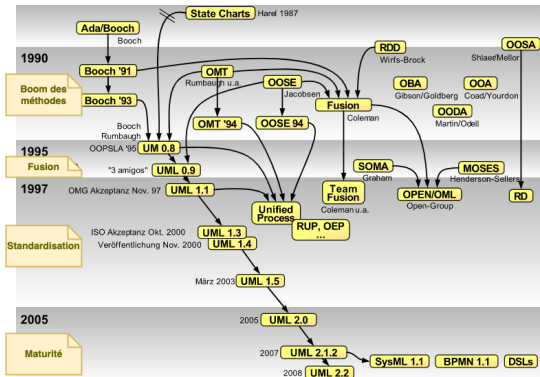
ATTENTION : orthographe des pictogrammes

UML est un **langage** comme le français : il y a donc un **"orthographe"** et une **"grammaire"**.

On n'utilise pas un □ à la place d'un ◇ ou une → à la place d'une -->

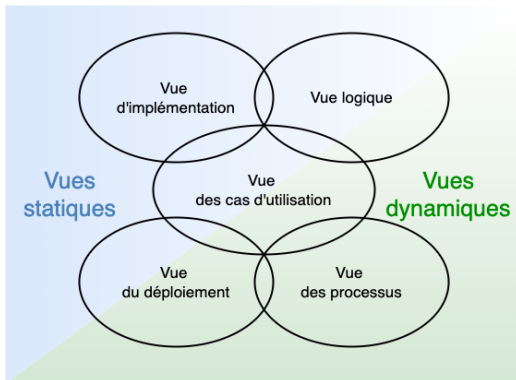
(bref) historique d'UML

- élaboré par *Rumbaugh*, *Booch* et *Jacobsen* (entre autre) à partir de 1995
- = fusion de différentes méthodes : OMT, OOSE, statechart, etc.
- Normalisé par l'OMG (Object Management Group)¹
- Première version officielle 1.1 en 1997
- La dernière version validée par l'OMG date de 2017 : 2.5.1



Facettes d'un système vs. UML

Un système (informatique) est **trop complexe** pour être décrit par un seul diagramme : il y a différentes facettes qui représentent chacune des aspects différents du système (statique, dynamique, temporel, etc.)



Des diagrammes spécifiques permettent de décrire tous ces aspects.

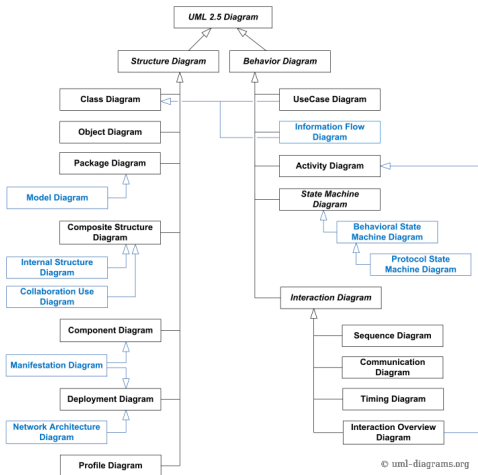
Diagrammes en UML 2.5

Diagrammes de structure (statiques)

- **Diagramme de classes**
- **Diagramme d'objets**
- **Diagramme de paquets**
- Diagramme de structure composite
- **Diagramme de composants**
- **Diagramme de déploiement**
- Diagramme de profils

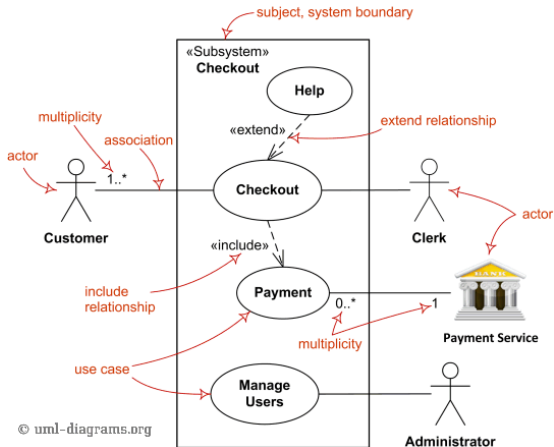
Diagrammes de comportement (dynamiques)

- Diagramme des cas d'utilisation
- Diagramme d'activité
- Diagramme états-transitions
- *Diagrammes d'interaction*
 - ▶ **Diagramme de séquence**
 - ▶ **Diagramme de communication**
 - ▶ Diagramme de temps
 - ▶ Diagramme global d'interaction



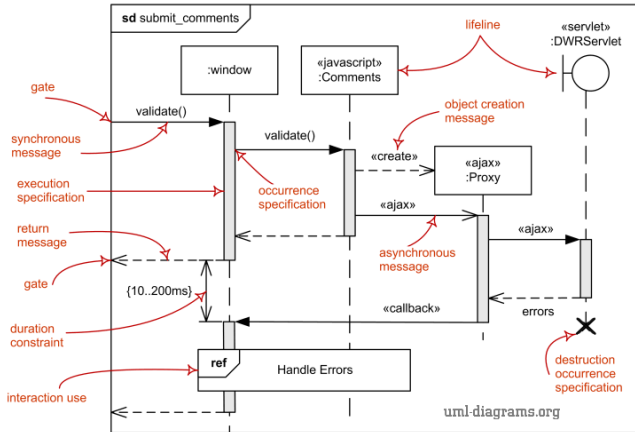
© uml-diagrams.org

Exemple de diagramme de cas d'utilisation

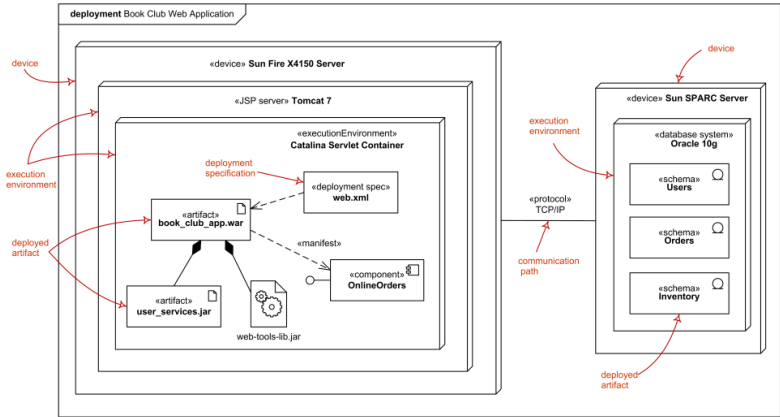


détaillé dans la ressource **R2.10 > Gestion de Projets informatiques** par Jean-Marie Mottu

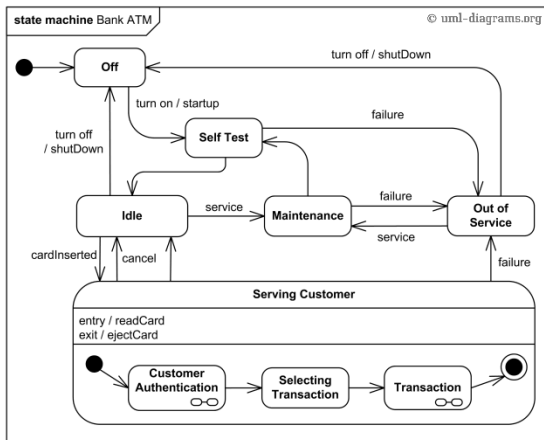
Exemple de diagramme de séquence



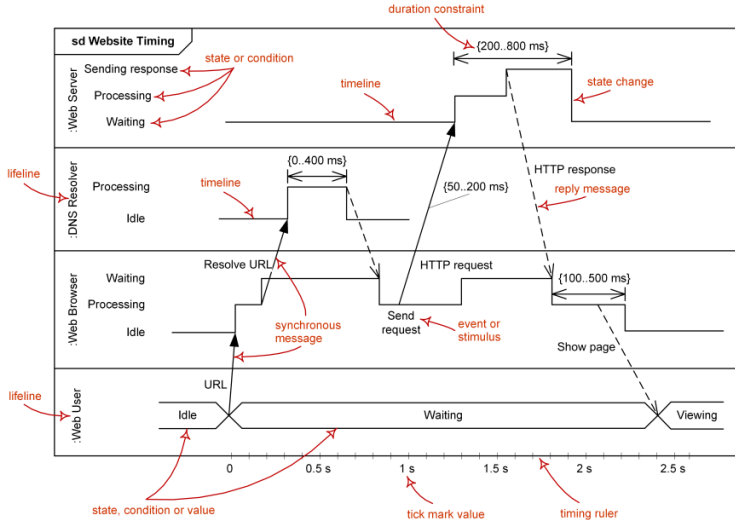
Exemple de diagramme de déploiement



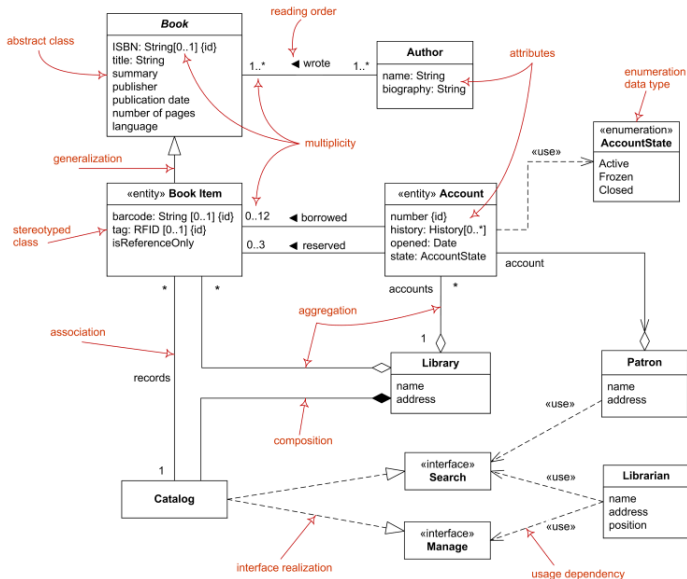
Exemple de diagramme états-transitions



Exemple de diagramme de temps



Exemple de diagramme de classes



- 1 Courte introduction à UML
- 2 Modéliser des classes grâce à UML

Rappel : qu'est-ce qu'une classe ?

= sorte de "moule"² pour définir des objets, qui précise

- les **propriétés** qui définissent la structure interne des objets (= les **attributs**)
- Les **interactions** qu'offrent les objets, les **comportements** possible pour les objets (= les **méthodes**)
- Les (éventuels) liens d'héritage
- ...

La classe **Citoyen**

- nom, prénom, date de naissance,
- numéro carte d'identité,
- photo,
- signature,
- ...



2. "patron", "plan", "schéma de construction", ...

Diagramme de classes UML

- Diagramme utilisé pour représenter **graphiquement** les **données métier** d'un système ainsi que leurs **relations**
- en termes de **classes** et d'**associations** (entre classes)
- Une classe décrit des **caractéristiques de base** liées ensemble sémantiquement (=attributs) ET des **interactions possibles** (= méthodes)
- Un objet est une **instance** d'une classe
- Une association décrit une **relation sémantique binaire** unissant certaines instances des classes considérées

ATTENTION : diagrammes de classes vs. MCD

Dans la ressource **R1.05 > intro aux BDDs et SQL** (Gilles Nachouki), on vous a présenté le **modèle conceptuel de données (MCD)** (i.e. entité-association), qui est très proche d'un diagramme de classes

Des règles de transformations existent.

Diagramme de classes UML

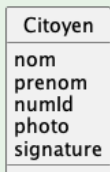
- Diagramme utilisé pour représenter **graphiquement** les **données métier** d'un système ainsi que leurs **relations**
- en termes de **classes** et d'**associations** (entre classes)
- Une classe décrit des **caractéristiques de base** liées ensemble sémantiquement (=attributs) ET des **interactions possibles** (= méthodes)
- Un objet est une **instance** d'une classe
- Une association décrit une **relation sémantique binaire** unissant certaines instances des classes considérées

ATTENTION : diagrammes de classes vs. MCD

Dans la ressource **R1.05 > intro aux BDDs et SQL** (Gilles Nachouki), on vous a présenté le **modèle conceptuel de données (MCD)** (i.e. entité-association), qui est très proche d'un diagramme de classes

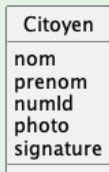
Des règles de transformations existent.

Exemple Citoyen : un premier diagramme de classes

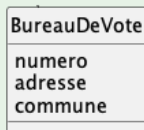


- 1 classe Citoyen qui regroupe les caractéristiques nom, prénom, numld, photo, ... définissant un "patron" de citoyen
- 1 classe BureauDeVote qui regroupe les caractéristiques numero, adresse, commune, ... définissant un "patron" de bureau de vote
- 1 association est inscrit qui caractérise une relation (binaire) entre Citoyen et Bureau de vote
elle indique qu'un citoyen donné peut être inscrit dans un bureau de vote et qu'un bureau de vote donné est le lieu d'inscription de plusieurs citoyens donnés

Exemple Citoyen : un premier diagramme de classes

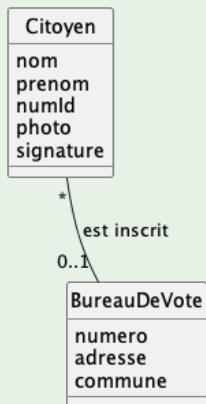


- 1 classe Citoyen qui regroupe les caractéristiques nom, prénom, numld, photo, ... définissant un "patron" de citoyen
- 1 classe BureauDeVote qui regroupe les caractéristiques numero, adresse, commune, ... définissant un "patron" de bureau de vote



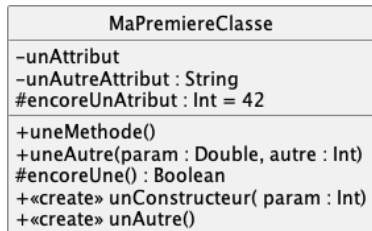
- 1 association est *inscrit* qui caractérise une relation (binaire) entre Citoyen et Bureau de vote
elle indique qu'un citoyen donné peut être inscrit dans un bureau de vote et qu'un bureau de vote donné est le lieu d'inscription de plusieurs citoyens donnés

Exemple Citoyen : un premier diagramme de classes



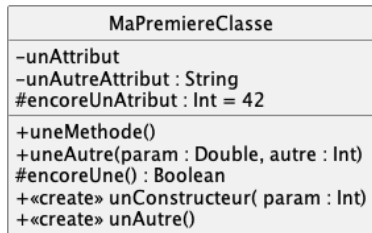
- 1 classe Citoyen qui regroupe les caractéristiques nom, prénom, numId, photo, ... définissant un "patron" de citoyen
- 1 classe BureauDeVote qui regroupe les caractéristiques numero, adresse, commune, ... définissant un "patron" de bureau de vote
- 1 association est inscrit qui caractérise une relation (binaire) entre Citoyen et Bureau de vote
*elle indique qu'un citoyen donné **peut être** inscrit dans un bureau de vote et qu'un bureau de vote donné est le lieu d'inscription de **plusieurs** citoyens donnés*

Décrire une classe en UML



- Une **boîte** rectangulaire
 - ▶ avec 1, 2 ou 3 compartiments
- Un **nom** de classe
 - ▶ = forme nominale
- Une liste d'**attributs** (optionnelle)
 - ▶ typés ou non
 - ▶ avec une initialisation possible
 - ▶ **ATTENTION** : uniquement des concepts/types simples
- Une liste de **méthodes** (optionnelle)
 - ▶ des paramètres sont possibles
 - ▶ des types de retour
 - ▶ les **constructeurs** sont à préciser par <<create>>
- Les **visibilités** (des attributs/des méthodes)
 - ▶ Encapsulation : l'élément est-il accessible par d'autres ou interne à la classe ?

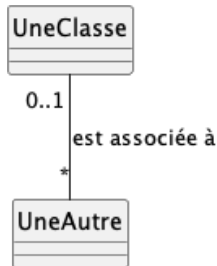
Décrire une classe en UML



+	publique
-	privée
#	protégée

- Une **boîte** rectangulaire
 - ▶ avec 1, 2 ou 3 compartiments
- Un **nom** de classe
 - ▶ = forme nominale
- Une liste d'**attributs** (optionnelle)
 - ▶ typés ou non
 - ▶ avec une initialisation possible
 - ▶ **ATTENTION** : uniquement des concepts/types simples
- Une liste de **méthodes** (optionnelle)
 - ▶ des paramètres sont possibles
 - ▶ des types de retour
 - ▶ les **constructeurs** sont à préciser par <<create>>
- Les **visibilités** (des attributs/des méthodes)
 - ▶ Encapsulation : l'élément est-il accessible par d'autres ou interne à la classe ?

Décrire une association en UML



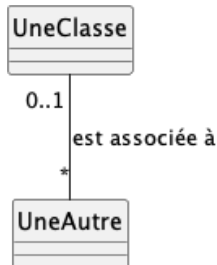
- Un **trait plein** reliant deux classes
- Un **nom** d'association
 - = forme verbale exprimant la sémantique de la relation
- Des **cardinalités**
 - = contrainte sur les instances en jeu dans la relation

1	un et un seul
0..1	zéro ou un
*	plusieurs
0..*	de 0 à plusieurs
1..*	de 1 à plusieurs
N..M	entre N et M

Attention

Nom d'association et cardinalités sont **obligatoires**

Décrire une association en UML



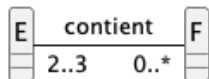
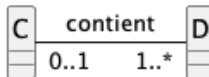
- Un **trait plein** reliant deux classes
- Un **nom** d'association
 - = forme verbale exprimant la sémantique de la relation
- Des **cardinalités**
 - = contrainte sur les instances en jeu dans la relation

1	un et un seul
0..1	zéro ou un
*	plusieurs
0..*	de 0 à plusieurs
1..*	de 1 à plusieurs
N..M	entre N et M

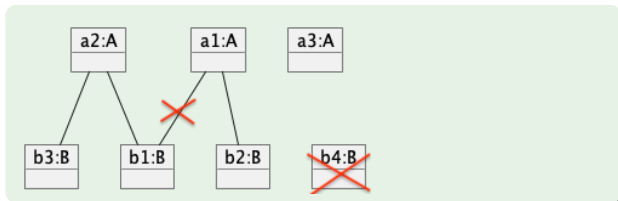
Attention

Nom d'association **et** cardinalités sont **obligatoires**

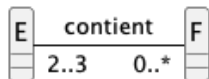
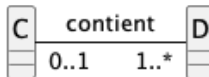
Les cardinalités



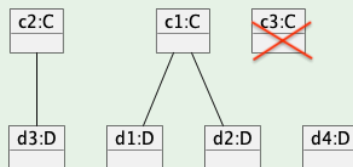
- cardinalité * = une instance de A contient **plusieurs** instances de B
- cardinalité 1 = une instance de B est contenue par **1 et 1 seule** instance de A
- cardinalité 1..* = une instance de C contient **plusieurs, mais au moins 1** instances de D
- cardinalité 0..1 = une instance de D est contenue par **0 ou 1** instance de C
- cardinalité 0..* = une instance de E contient **plusieurs, mais possiblement 0** instances de F
- cardinalité 2..3 = une instance de F est contenue par **entre 2 et 3** instances de E



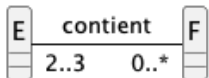
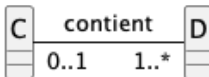
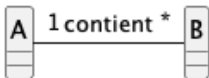
Les cardinalités



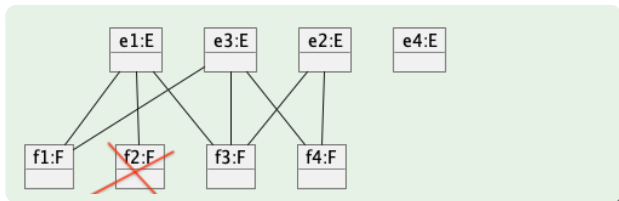
- cardinalité * = une instance de A contient **plusieurs** instances de B
- cardinalité 1 = une instance de B est contenue par **1 et 1 seule** instance de A
- cardinalité 1..* = une instance de C contient **plusieurs, mais au moins 1** instances de D
- cardinalité 0..1 = une instance de D est contenue par **0 ou 1** instance de C
- cardinalité 0..* = une instance de E contient **plusieurs, mais possiblement 0** instances de F
- cardinalité 2..3 = une instance de F est contenue par **entre 2 et 3** instances de E



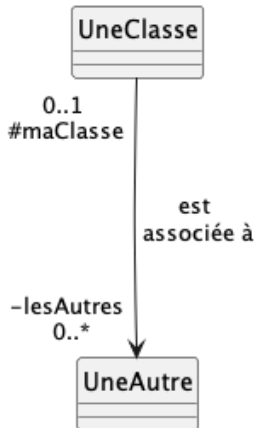
Les cardinalités



- cardinalité * = une instance de A contient **plusieurs** instances de B
- cardinalité 1 = une instance de B est contenue par **1 et 1 seule** instance de A
- cardinalité 1..* = une instance de C contient **plusieurs, mais au moins 1** instances de D
- cardinalité 0..1 = une instance de D est contenue par **0 ou 1** instance de C
- cardinalité 0..* = une instance de E contient **plusieurs, mais possiblement 0** instances de F
- cardinalité 2..3 = une instance de F est contenue par **entre 2 et 3** instances de E



Décrire une association en UML : précisions possible

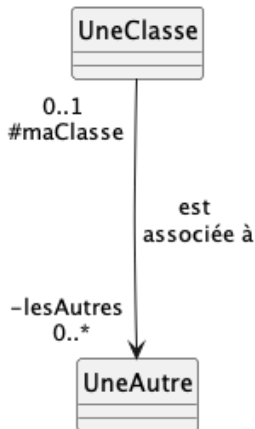


- le **rôle** joué par une classe dans l'autre classe peut être précisé
 - ▶ la visibilité peut aussi être précisée
- Une **flèche** au bout de l'association indique la **navigabilité**, c-à-d une association **unidirectionnelle**
 - ▶ sans navigabilité, l'association est bidirectionnelle (**par défaut**)

Attention

Rôle et navigabilité sont **facultatifs**

Décrire une association en UML : précisions possible



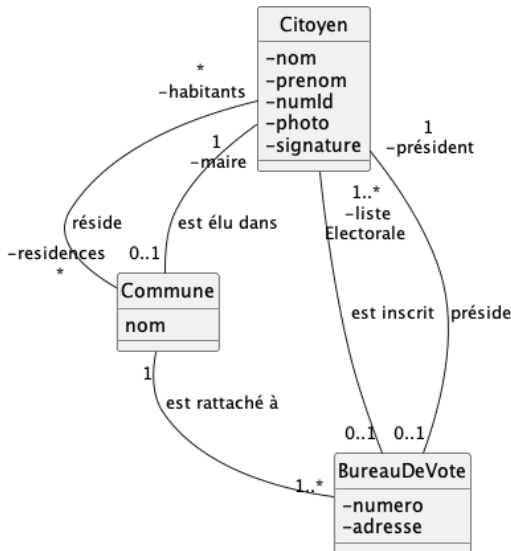
- le **rôle** joué par une classe dans l'autre classe peut être précisé
 - ▶ la visibilité peut aussi être précisée
- Une **flèche** au bout de l'association indique la **navigabilité**, c-à-d une association **unidirectionnelle**
 - ▶ sans navigabilité, l'association est bidirectionnelle (**par défaut**)

Attention

Rôle **et** navigabilité sont **facultatifs**

Exemple Citoyen plus complexe

Des classes peuvent être en jeu dans **plusieurs** associations



(au moins) deux diagrammes de classes (1)

1 niveau *Analyse* : on précise **uniquement**

- ▶ les entités métiers (= classes + attributs)
- ▶ et leurs relations (= les associations + les cardinalités)

But : capturer le domaine métier

- x les constructeurs,
- x les méthodes (sauf si vraiment pertinent),
- x les types pour tous les attributs (sauf si évident)

(au moins) deux diagrammes de classes (1)

1 niveau *Analyse* : on précise **uniquement**

- ▶ les entités métiers (= classes + attributs)
- ▶ et leurs relations (= les associations + les cardinalités)

But : capturer le domaine métier

de quoi on va parler ?

- x les constructeurs,
- x les méthodes (sauf si vraiment pertinent),
- x les types pour tous les attributs (sauf si évident)

(au moins) deux diagrammes de classes (1)

① niveau **Analyse** : on précise **uniquement**

- ▶ les entités métiers (= classes + attributs)
- ▶ et leurs relations (= les associations + les cardinalités)

But : capturer le domaine métier

de quoi on va parler ?

On ne précise pas

- x les constructeurs,
- x les méthodes (sauf si vraiment pertinent),
- x les types pour tous les attributs (sauf si évident)

(au moins) deux diagrammes de classes (2)

❶ niveau **Analyse** : ...

de quoi on va parler ?

❷ niveau **Conception détaillée** : on prépare l'implémentation

- ▶ on précise les **types** de tous les attributs,
- ▶ on précise le **constructeur** de chaque classe,
- ▶ on précise toutes les méthodes,
- ▶ certaines classes peuvent disparaître,
- ▶ des classes peuvent apparaître (des classes "outils")
- ▶ On précise **navigabilité** et **rôles** des associations
- ▶ ...

But : obtenir un "plan" précis à implémenter

Diagramme de classes vers Modèle relationnel

une autre **conception détaillée** partant d'un diagramme de classes d'analyse peut être un **modèle relationnel de données** si l'implémentation s'oriente vers une BDD

(au moins) deux diagrammes de classes (2)

❶ niveau **Analyse** : ...

de quoi on va parler ?

❷ niveau **Conception détaillée** : on *prépare* l'implémentation

- ▶ on précise les **types** de tous les attributs,
- ▶ on précise le **constructeur** de chaque classe,
- ▶ on précise toutes les méthodes,
- ▶ certaines classes peuvent disparaître,
- ▶ des classes peuvent apparaître (des classes "outils")
- ▶ On précise **navigabilité** et **rôles** des associations
- ▶ ...

But : obtenir un "plan" précis à implémenter

Diagramme de classes vers Modèle relationnel

une autre **conception détaillée** partant d'un diagramme de classes d'analyse peut être un **modèle relationnel de données** si l'implémentation s'oriente vers une BDD

(au moins) deux diagrammes de classes (2)

❶ niveau **Analyse** : ...

de quoi on va parler ?

❷ niveau **Conception détaillée** : on *prépare* l'implémentation

- ▶ on précise les **types** de tous les attributs,
- ▶ on précise le **constructeur** de chaque classe,
- ▶ on précise toutes les méthodes,
- ▶ certaines classes peuvent disparaître,
- ▶ des classes peuvent apparaître (des classes "outils")
- ▶ On précise **navigabilité** et **rôles** des associations
- ▶ ...

comment on va
le réaliser ?

But : obtenir un "plan" précis à implémenter

Diagramme de classes vers Modèle relationnel

une autre **conception détaillée** partant d'un diagramme de classes d'analyse peut être un **modèle relationnel de données** si l'implémentation s'oriente vers une BDD

(au moins) deux diagrammes de classes (2)

❶ niveau **Analyse** : ...

de quoi on va parler ?

❷ niveau **Conception détaillée** : on prépare l'implémentation

- ▶ on précise les **types** de tous les attributs,
- ▶ on précise le **constructeur** de chaque classe,
- ▶ on précise toutes les méthodes,
- ▶ certaines classes peuvent disparaître,
- ▶ des classes peuvent apparaître (des classes "outils")
- ▶ On précise **navigabilité** et **rôles** des associations
- ▶ ...

comment on va
le réaliser ?

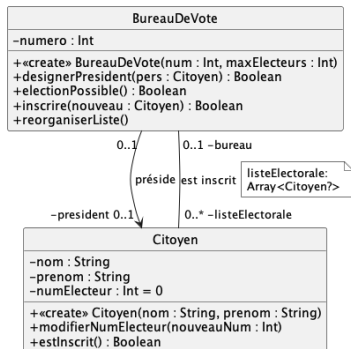
But : obtenir un "plan" précis à implémenter

Diagramme de classes vers Modèle relationnel

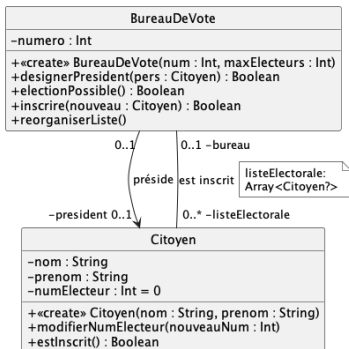
une autre **conception détaillée** partant d'un diagramme de classes d'analyse peut être un **modèle relationnel de données** si l'implémentation s'oriente vers une BDD

Exemple Citoyen : conception détaillée (orientée objet)

- On précise les types des attributs
 - ▶ cela dépend du langage cible, ici **Kotlin**
- On ajoute des **constructeurs**
 - ▶ Notation :
`<<create>> NomDeLaClasse(parametres)`
 - ▶ Les paramètres sont les valeurs nécessaires pour valuer une instance de la classe
- On ajoute les **méthodes** nécessaires
 - ▶ les signatures doivent être compatibles avec le langage cible
- On précise les **navigabilités** et les **rôles** des associations
 - ▶ préférez des associations **unidirectionnelles**
- On précise comment seront implémentées les **cardinalités** *, 0..*, 1..*
 - ▶ cela dépend du langage cible
 - ▶ dans une **Note** ou en dehors du diagramme de classes

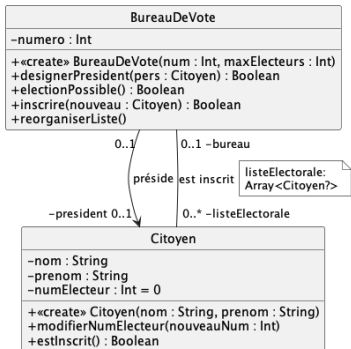


Exemple Citoyen : conception détaillée (orientée objet)



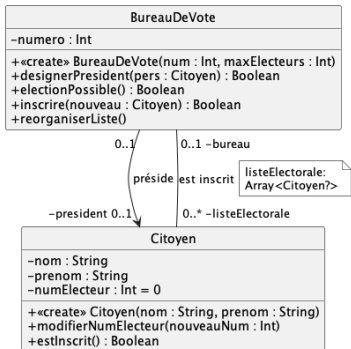
- On précise les types des attributs
 - ▶ cela dépend du langage cible, ici **Kotlin**
- On ajoute des **constructeurs**
 - ▶ Notation :
`<<create>> NomDeLaClasse(parametres)`
 - ▶ Les paramètres sont les valeurs nécessaires pour valuer une instance de la classe
- On ajoute les **méthodes** nécessaires
 - ▶ les signatures doivent être compatibles avec le langage cible
- On précise les **navigabilités** et les **rôles** des associations
 - ▶ préférez des associations **unidirectionnelles**
- On précise comment seront implémentées les **cardinalités** `*`, `0..*`, `1..*`
 - ▶ cela dépend du langage cible
 - ▶ dans une **Note** ou en dehors du diagramme de classes

Exemple Citoyen : conception détaillée (orientée objet)



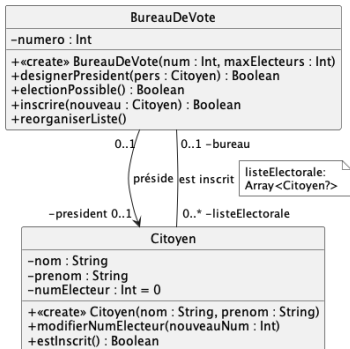
- On précise les types des attributs
 - cela dépend du langage cible, ici **Kotlin**
- On ajoute des **constructeurs**
 - Notation : `<<create>> NomDeLaClasse(parametres)`
 - Les paramètres sont les valeurs nécessaires pour valuer une instance de la classe
- On ajoute les **méthodes** nécessaires
 - les signatures doivent être compatibles avec le langage cible
- On précise les **navigabilités** et les **rôles** des associations
 - préférez des associations **unidirectionnelles**
- On précise comment seront implémentées les **cardinalités** `*`, `0..*`, `1..*`
 - cela dépend du langage cible
 - dans une **Note** ou en dehors du diagramme de classes

Exemple Citoyen : conception détaillée (orientée objet)



- On précise les types des attributs
 - cela dépend du langage cible, ici **Kotlin**
- On ajoute des **constructeurs**
 - Notation :
`<<create>> NomDeLaClasse(parametres)`
 - Les paramètres sont les valeurs nécessaires pour valuer une instance de la classe
- On ajoute les **méthodes** nécessaires
 - les signatures doivent être compatibles avec le langage cible
- On précise les **navigabilités** et les **rôles** des associations
 - **préférez** des associations **unidirectionnelles**
- On précise comment seront implémentées les **cardinalités** `*`, `0..*`, `1..*`
 - cela dépend du langage cible
 - dans une **Note** ou en dehors du diagramme de classes

Exemple Citoyen : conception détaillée (orientée objet)



- On précise les types des attributs
 - cela dépend du langage cible, ici **Kotlin**
- On ajoute des **constructeurs**
 - Notation :
`<<create>> NomDeLaClasse(parametres)`
 - Les paramètres sont les valeurs nécessaires pour valuer une instance de la classe
- On ajoute les **méthodes** nécessaires
 - les signatures doivent être compatibles avec le langage cible
- On précise les **navigabilités** et les **rôles** des associations
 - **préférez** des associations **unidirectionnelles**
- On précise comment seront implémentées les **cardinalités** `*`, `0..*`, `1..*`
 - cela dépend du langage cible
 - dans une **Note** ou en dehors du diagramme de classes

Aller plus loin (en UML)

De nombreux aspects concernant les diagrammes de classes UML n'ont pas encore été abordés

- des précisions concernant les associations
 - ▶ agrégation / composition
 - ▶ contraintes entre association
 - ▶ classes-association
- **les relations d'héritage**
 - ▶ classes abstraites et généralisation
 - ▶ interfaces et réalisation
- les attributs / les méthodes de classes
- les relations de dépendances (faibles)
- les diagrammes d'objets / de paquetages

[CM dédié]